❺ If authorization failed, returns a Forbidden result

❻ If authorization was successful, continues displaying the Razor Page

`IAuthorizationService` exposes an `AuthorizeAsync` method, which requires three things to authorize the request:

- The `ClaimsPrincipal` user object, exposed on the `PageModel` as `User`
- The resource being authorized: `Recipe`
- The policy to evaluate: `"CanManageRecipe"`

The authorization attempt returns an `AuthorizationResult` object, which indicates whether the attempt was successful via the `Succeeded` property. If the attempt wasn't successful, you should return a new `ForbidResult`, which is converted to an HTTP `403 Forbidden` response or redirects the user to the "access denied" page, depending on whether you're building a traditional web app or an API app.

NOTE As mentioned in section 24.2.2, which type of response is generated depends on which authentication services are configured. The default Identity configuration, used by Razor Pages, generates redirects. API apps typically generate HTTP `401` and `403` responses instead.

You've configured the imperative authorization in the Edit.cshtml Razor Page itself, but you still need to define the `"CanManageRecipe"` policy that you use to authorize the user. This is the same process as for declarative authorization, so you have to do the following:

- Create a policy in Program.cs by calling `AddAuthorization()`.